

Unit 2

DATA PREPROCESSING

Data Manipulation

1. Handling Missing Data

Aim

To identify missing values in a dataset, fill missing values using the median, and remove rows containing missing salary values.

Algorithm

1. Import the Pandas and NumPy libraries.
2. Create a DataFrame containing missing values.
3. Check for missing values using `isnull().sum()`.
4. Fill missing values in the **Age** column using the median.
5. Remove rows with missing values in the **Salary** column using `dropna()`.
6. Display the cleaned DataFrame.

Code:

```
import pandas as pd
import numpy as np

# Sample data with missing entries
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, np.nan, 30, 22],
    'Salary': [50000, 60000, np.nan, 45000]
}
df = pd.DataFrame(data)

# 1. Check for missing values
print("Missing count:\n", df.isnull().sum())

# 2. Fill missing Age with the column median
df['Age'] = df['Age'].fillna(df['Age'].median())

# 3. Drop rows where Salary is missing
df = df.dropna(subset=['Salary'])
print("\nCleaned DataFrame:\n", df)
```

OUTPUT

1. Handling Missing Data

[10]

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, np.nan, 30, 22],
    'Salary': [50000, 60000, np.nan, 45000]
}
df = pd.DataFrame(data)

print("Missing count:\n", df.isnull().sum())

df['Age'] = df['Age'].fillna(df['Age'].median())

df = df.dropna(subset=['Salary'])
print("\nCleaned DataFrame:\n", df)
```

```
... Missing count:
   Name      0
   Age      1
   Salary    1
   dtype: int64

Cleaned DataFrame:
   Name  Age  Salary
0  Alice  25.0  50000.0
1   Bob  25.0  60000.0
3  David  22.0  45000.0
```

Result

Thus, the missing values were successfully identified, the missing age values were filled with the median, and rows containing missing salary values were removed from the dataset.

2. Filtering Rows Based on Multiple Conditions

Aim

To filter records from a dataset based on multiple conditions using Pandas.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing employee details.
3. Apply multiple conditions to filter employees:
 - Department is **IT**.
 - Experience is greater than **5 years**.
4. Display the filtered records.

Code:

```
import pandas as pd
```

```
data = {  
    'Employee': ['Amy', 'John', 'Raj', 'Kevin', 'Sora'],  
    'Department': ['HR', 'IT', 'IT', 'Sales', 'IT'],  
    'Experience': [3, 7, 1, 5, 6]  
}  
df = pd.DataFrame(data)
```

```
# Filter for employees in IT with more than 5 years of experience  
filtered_df = df[(df['Department'] == 'IT') & (df['Experience'] > 5)]  
print(filtered_df)
```

OUTPUT

2. Filtering Rows Based on Multiple Conditions

```
[14] import pandas as pd  
data = {  
    'Employee': ['Amy', 'John', 'Raj', 'Kevin', 'Sora'],  
    'Department': ['HR', 'IT', 'IT', 'Sales', 'IT'],  
    'Experience': [3, 7, 1, 5, 6]  
}  
df = pd.DataFrame(data)  
  
filtered_df = df[(df['Department'] == 'IT') & (df['Experience'] > 5)]  
print(filtered_df)
```

	Employee	Department	Experience
1	John	IT	7
4	Sora	IT	6

Result

Thus, the employee records satisfying multiple conditions were successfully filtered and displayed.

3. Aggregating and Grouping Data

Aim

To group data based on stores and calculate the total sales and average number of employees.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing store details.
3. Group the data using the **Store** column.
4. Calculate:
 - Total sales using `sum()`.
 - Average employees using `mean()`.
5. Display the aggregated results.

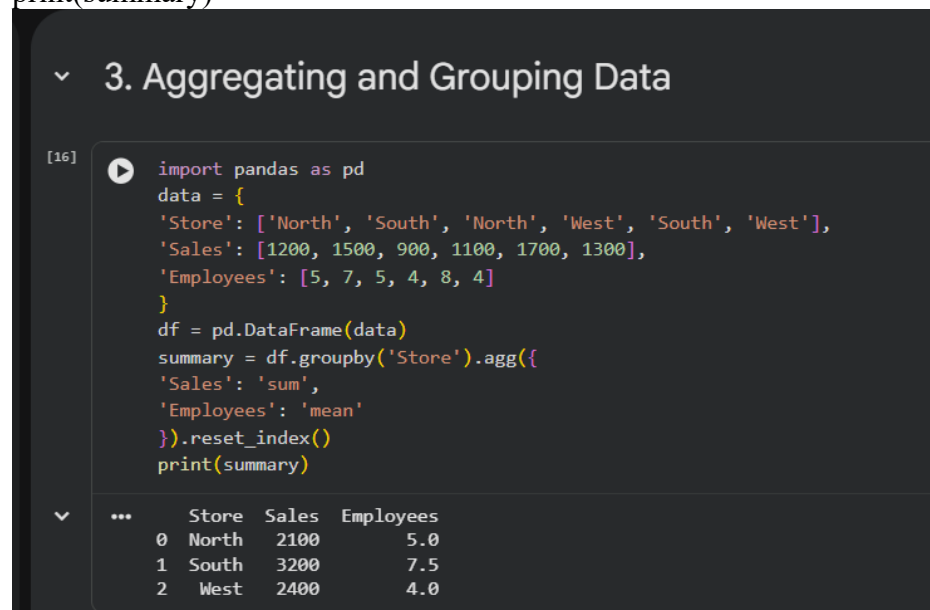
Code:

```
import pandas as pd
```

```
data = {  
    'Store': ['North', 'South', 'North', 'West', 'South', 'West'],  
    'Sales': [1200, 1500, 900, 1100, 1700, 1300],  
    'Employees': [5, 7, 5, 4, 8, 4]  
}  
df = pd.DataFrame(data)
```

```
# Group by Store and calculate total sales and average employees  
summary = df.groupby('Store').agg({  
    'Sales': 'sum',  
    'Employees': 'mean'  
}).reset_index()
```

```
print(summary)
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The title of the notebook is "3. Aggregating and Grouping Data". The code cell [16] contains the following Python code:

```
import pandas as pd  
data = {  
    'Store': ['North', 'South', 'North', 'West', 'South', 'West'],  
    'Sales': [1200, 1500, 900, 1100, 1700, 1300],  
    'Employees': [5, 7, 5, 4, 8, 4]  
}  
df = pd.DataFrame(data)  
summary = df.groupby('Store').agg({  
    'Sales': 'sum',  
    'Employees': 'mean'  
}).reset_index()  
print(summary)
```

Below the code cell, the output is displayed as a table:

	Store	Sales	Employees
0	North	2100	5.0
1	South	3200	7.5
2	West	2400	4.0

Result

Thus, the data was successfully grouped by store, and the total sales and average number of employees for each store were calculated.

4. Python Program Using describe()

Aim

To generate summary statistics for numeric and categorical data using the describe() function.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing numeric and categorical columns.
3. Use describe() to obtain summary statistics for numeric columns.
4. Use describe(include='object') to obtain summary statistics for categorical columns.
5. Display the results.

code:

```
import pandas as pd
data = {
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 75000, 90000, 110000],
    'Department': ['HR', 'IT', 'IT', 'Marketing', 'HR']
}
df = pd.DataFrame(data)
print("--- Numeric Summary ---")
print(df.describe())
print("\n--- Categorical Summary ---")
print(df.describe(include='object'))
```

4. Python Program Using describe()

```
[18] import pandas as pd
data = {
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 75000, 90000, 110000],
    'Department': ['HR', 'IT', 'IT', 'Marketing', 'HR']
}
df = pd.DataFrame(data)
print("--- Numeric Summary ---")
print(df.describe())
print("\n--- Categorical Summary ---")
print(df.describe(include='object'))
```

```
--- Numeric Summary ---
      Age      Salary
count  5.000000    5.000000
mean   35.000000   77000.000000
std     7.905694   23874.672773
min    25.000000   50000.000000
25%    30.000000   60000.000000
50%    35.000000   75000.000000
75%    40.000000   90000.000000
max    45.000000  110000.000000

--- Categorical Summary ---
      Department
count          5
unique         3
top           HR
freq          2
```

Result

Thus, the statistical summary of both numeric and categorical data was successfully generated using the describe() function.

5. head() and tail() in Python

Aim

To display the first and last records of a dataset using the head() and tail() functions.

Algorithm

1. Import the Pandas library.
2. Create a sample DataFrame.
3. Display the first five rows using head().
4. Display the first two rows using head(2).
5. Display the last three rows using tail(3).
6. Display the output.

Code:

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace'],
    'Score': [85, 92, 78, 95, 88, 72, 91]
}
df = pd.DataFrame(data)
print(df.head())
print(df.head(2))
print(df.tail(3))
```

OUTPUT:

```
5. head() and tail() in Python

[20] import pandas as pd
data = {

    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace'],
    'Score': [85, 92, 78, 95, 88, 72, 91]
}
df = pd.DataFrame(data)
print(df.head())

print(df.head(2))

print(df.tail(3))
```

	Name	Score
0	Alice	85
1	Bob	92
2	Charlie	78
3	David	95
4	Eva	88

	Name	Score
0	Alice	85
1	Bob	92

	Name	Score
4	Eva	88
5	Frank	72
6	Grace	91

Result

Thus, the first and last records of the dataset were successfully displayed using the head() and tail() functions